Make Utility:

The purpose of the **make** utility is to determine automatically which pieces of a large program need to be re-compiled, and issue the commands necessary to recompile them. You can use make with any programming language whose compiler can be run with a shell command. In fact, make is not limited to programs. You can use it to describe any task where some files must be updated automatically from others whenever the others change.
To prepare to use make, you must write a file called the makefile that describes the relationships among files in your program, and the states the commands for updating each file. In a program, typically the executable file is updated from object files, which are in turn made by compiling source files.

Make gets its knowledge of how to build your program from a file called the makefile, which lists each of the non-source files and how to compute it from other files. When you write a program, you should write a makefile for it, so that it is possible to use Make to build and install the program.

Capabilities of Make

Make enables the end user to build and install your package without knowing the details of how that is done -- because these details are recorded in the makefile that you supply.

Make figures out automatically which files it needs to update, based on which source files have changed. It also automatically determines the proper order for updating files, in case one non-source file depends on another non-source file.

As a result, if you change a few source files and then run Make, *it does not need to recompile all of your program.* It updates only those non-source files that depend directly or indirectly on the source files that you changed.

Make is not limited to any particular language. For each non-source file in the program, the makefile specifies the shell commands to compute it. These shell commands can run a compiler

Vectorindia

to produce an object file, the linker to produce an executable, ar to update a library, or TeX or Makeinfo to format documentation.

Make is not limited to building a package. You can also use Make to control installing or deinstalling a package, generate tags tables for it, or anything else you want to do often enough to make it worth while writing down how to do it.

The Makefile :

A file called makefile tells the make utility in a structured manner which source and object files depend on other files. It also defines the commands required to compile and link the files.

Each file to build, or step to perform, is called a *target*. Each entry in a makefile is a rule expressing a target object's dependencies and the commands needed to build or make that object. The structure of a rule in the makefile is:

```
target:
dependencies-list

TAB        build-
commands
```

For the dependencies, each entry starts with a line that names the target file, followed by all the files the target depends on. For the build commands, each entry has one or more subsequent lines that specify the Bourne shell commands that will build the target file for this entry. Each of these command lines must be indented by a tab character.

Multiple makefiles in a Directory:

Normally standard version of make looks first for a file called makefile in the current directory. If that doesn't exist, it looks for a file called Makefile. When we need to select specific make file from multiple makefiles we use -f option example if you

Vector India

want make command to choose a makefile_XYZ instead of makefile , then you can call it by -f options .

Eg. make -f makefil_XYZ

then make executes the commands present in makefile_XYZ .

Note :

Reference Books : Taken contents and diagrams from various websites.

Vector India